

Series I – Article I.6: Consequences and Implications – $P = NP$ Theoretically but Not Practically

Christian Kithima
Independent Researcher, Kinshasa
christiankithimaomba@gmail.com
WhatsApp: +243995176701
Telegram: +243818136082
ORCID: 0009-0003-3829-8519

GitHub: <https://github.com/christiankithimaomba-cyber/spectral-reduction-framework>

May 2026

Abstract

Article I.5 established that $P = NP$ by constructing a spectral Hamiltonian whose ground state extraction (D-RSP) solves SAT in polynomial time. The algorithm is correct and certified in Lean4. However, the complexity analysis reveals constants of astronomical size: the bound on the spectral gap $\gamma(H) \geq 1/(8n^2)$ leads to a number of power iterations $T = O(n^2 \log n)$, the MPS bond dimension $\chi = O(n^c)$ with a large constant c , and the overall runtime is $O(n^{3c+4} \log n)$ with an implicit constant that is likely huge (e.g., exceeding 10^{100}). For any instance of SAT that fits in the observable universe, the algorithm would not terminate before the heat death of the universe. Therefore, while $P = NP$ is true mathematically, it has no practical consequence for cryptography or optimisation. This article explains why the cryptographic community need not panic, and why the spectral proof is a purely theoretical achievement.

Contents

1	Introduction	2
2	The distinction between polynomial time and practical efficiency	2
3	Why the constants are huge	2
4	Theoretical proof versus practical algorithm: a crucial nuance	3
4.1	Theoretical proof vs. practical algorithm	3
4.2	Advantages and disadvantages of a theoretical proof	3
4.3	What does the community really seek?	4
4.4	Reframing the problem: it is about constants, not existence	4
4.5	Why the community thought $P \neq NP$	4
4.6	Conclusion on the nuance	4
5	Implications for cryptography	4
5.1	RSA and ECC remain secure in practice	4
5.2	Theoretical impact only	4

6 Spectral cryptography – a mathematical curiosity	5
6.1 Principles of spectral cryptography	5
6.2 Encryption protocol	5
6.3 Decryption protocol	5
6.4 Security analysis	6
6.5 Implementation of spectral cryptography (Python)	6
7 Spectral factorisation (Python)	6
8 Conclusion	6

1 Introduction

Article I.5 established that $P = NP$ by constructing a spectral Hamiltonian whose ground state extraction (D-RSP) solves SAT in polynomial time. The algorithm is correct and certified in Lean4. However, the complexity analysis reveals constants of astronomical size: the bound on the spectral gap $\gamma(H) \geq 1/(8n^2)$ leads to a number of power iterations $T = O(n^2 \log n)$, the MPS bond dimension $\chi = O(n^c)$ with a large constant c , and the overall runtime is $O(n^{3c+4} \log n)$ with an implicit constant that is likely huge (e.g., exceeding 10^{100}). For any instance of SAT that fits in the observable universe, the algorithm would not terminate before the heat death of the universe. Therefore, while $P = NP$ is true mathematically, it has no practical consequence for cryptography or optimisation. This article explains why the cryptographic community need not panic, and why the spectral proof is a purely theoretical achievement.

2 The distinction between polynomial time and practical efficiency

A polynomial time algorithm is one whose worst-case runtime is bounded by $C \cdot n^k$ for some constants C, k . If C is astronomically large (e.g., $C = 10^{1000}$), the algorithm is unusable in practice. Classical examples include:

- The AKS primality test, which is polynomial but too slow for large numbers compared to probabilistic tests.
- The first polynomial algorithm for linear programming (Khachiyan’s ellipsoid method), which was impractical.

The D-RSP algorithm falls into this category: its constants are astronomical due to the deep potential M^2 , the logarithmic area law constants, and the required precision for power iteration.

3 Why the constants are huge

1. The spectral gap $\gamma(H) \geq 1/(8n^2)$ is polynomially small. The number of power iterations needed to achieve error δ is proportional to $1/\gamma$, i.e., $O(n^2)$. For $n = 100$, $n^2 = 10^4$, which is acceptable. But the hidden constant in the convergence bound may be enormous.
2. The MPS bond dimension $\chi = O(n^c)$ grows with a constant c that depends on the logarithmic area law. The Brandão–Horodecki theorem gives an existence bound, not an explicit small constant. In the worst case, c could be as large as 10^5 .
3. The compression algorithm (SVD truncation) requires high precision to maintain the amplitude gap. This precision translates into a large multiplicative factor.

Thus, even for $n = 50$, the algorithm would require an unimaginable number of operations.

4 Theoretical proof versus practical algorithm: a crucial nuance

The proof that $P = NP$ given in this series is a theoretical proof. It establishes the existence of a deterministic polynomial-time algorithm for SAT (the D-RSP algorithm), but the constants involved are astronomical. This section clarifies the distinction between a theoretical proof and a practical algorithm, and explains why the community’s long-standing belief that $P \neq NP$ was based on a hidden assumption.

4.1 Theoretical proof vs. practical algorithm

Criterion	Theoretical proof	Practical algorithm
Goal	Establish existence of algorithm	Provide a usable implementation
Complexity	Polynomial in worst case	Polynomial with small constants and low overhead
Example	AKS primality test (theoretical)	Miller–Rabin primality test (practical, probabilistic)
Impact	Changes fundamental understanding	Changes industrial practice

A theoretical proof says: “There exists a polynomial-time algorithm.” It does not say: “Here it is, it runs in 0.1 seconds.” In the case of the spectral proof of $P = NP$, the algorithm exists but its constants are astronomical – due to the spectral gap bound $\gamma(H) \geq 1/(8n^2)$, the MPS bond dimension $\chi = O(n^c)$ with a large exponent c , and the required precision for power iteration. Consequently, the algorithm is useless in practice, yet it fulfills the definition of polynomial time.

4.2 Advantages and disadvantages of a theoretical proof

Advantages:

- Closes the logical debate: We now know that $P = NP$ is mathematically true. No one can claim $P \neq NP$ without refuting the proof.
- Opens new directions: Researchers can now focus on improving the constants or finding more efficient algorithms, knowing that the theoretical barrier has been overcome.
- Credibility: The Lean4 certification guarantees that there is no logical error.
- Conceptual impact: Forces the community to reconsider its prejudices. Many believed $P \neq NP$ because they assumed that a polynomial algorithm would necessarily be efficient enough to break cryptography. That assumption is false.

Disadvantages:

- No immediate practical impact: RSA and ECC remain secure because the algorithm is far too slow.
- Community resistance: Some may say “this is not a real proof” because it is not constructive in the practical sense. This is a mistake: it is constructive in the mathematical sense (it explicitly describes the algorithm and its correctness).
- Astronomical constants: Experimentation is impossible; one cannot test the algorithm on real instances.

4.3 What does the community really seek?

Most theoretical computer scientists and mathematicians seek a theoretical proof, because that is what advances fundamental knowledge. The P vs NP question is foremost about the nature of computation, not about practical efficiency. Yet, many believed $P \neq NP$ because they thought a polynomial algorithm for SAT would necessarily be efficient (or at least efficient enough to break cryptography). This belief is erroneous. A polynomial algorithm can be terribly slow.

4.4 Reframing the problem: it is about constants, not existence

The spectral proof shows that the real barrier is not logical ($P \neq NP$) but practical (astronomical constants). The distinction between “easy” and “hard” is not an absolute dichotomy; it is a continuum where the hidden constant in the “O” plays a crucial role. Thus, the community has focused on the wrong question. It is not the existence of a polynomial algorithm that is at stake – it is the size of the constants. The true challenge is to reduce those constants, not to prove impossibility.

4.5 Why the community thought $P \neq NP$

Decades of research failed to produce an efficient polynomial algorithm for SAT, and results like non-relativisation suggested that standard methods would fail. What was not envisioned is that a polynomial algorithm could exist but be too slow to be useful. This is a cognitive bias: the implicit assumption that “polynomial = practical”. The spectral proof breaks this bias.

4.6 Conclusion on the nuance

- Theoretical proof: It exists, it is correct, it changes our understanding.
- Practical algorithm: It does not yet exist for $P = NP$, and the present work suggests that it may be beyond current (and possibly future) technology.
- The community must now recognise that $P = NP$ is true, but that this has no practical consequence for cryptography or optimisation.

Key takeaway: “Every problem is theoretically solvable within the spectral framework (as proved in Series I). However, the astronomical size of certain constants means that the problem is not solvable in practice due to lack of adequate hardware. Consequently, the complexity problem should be reoriented toward the size of constants, not toward a theoretical impossibility.”

5 Implications for cryptography

5.1 RSA and ECC remain secure in practice

Although factoring and discrete logarithm are in P (by reduction to SAT), the D-RSP algorithm cannot factor a 2048-bit RSA modulus because the SAT instance would have millions of variables, and the astronomical constants would make the algorithm run for an astronomical time. Hence, no practical attack is enabled.

5.2 Theoretical impact only

The result forces a rethinking of complexity theory: the P vs NP problem is settled in the positive direction. But the separation between “easy” (polynomial) and “hard” (exponential) was never about practical feasibility – it was about asymptotic growth. The algorithm demonstrates that polynomial can be incredibly slow.

6 Spectral cryptography – a mathematical curiosity

6.1 Principles of spectral cryptography

Spectral cryptography relies on representing keys as spectral signatures in Hilbert space $\mathcal{H}_n = \ell^2(\{0, 1\}^n)$, and on the difficulty of recovering a signature from observing geodesic trajectories.

Definition 6.1 (Spectral key). *A spectral key is a vector $\lambda_K \in \mathcal{H}_n$ (the spectral signature). This key can be derived from any data: text, image, sound, fingerprint, etc. We assume an injection $\text{encode} : \mathcal{K} \rightarrow \mathcal{H}_n$ where $\mathcal{K}$ is the set of possible keys.*

To each key K we associate a Hamiltonian H_K constructed from λ_K . For example, we may take $\hat{V}_K$ diagonal with $(\hat{V}_K)_{xx} = \|\lambda_K - \delta_x\|^2$, so that the ground state ψ_K is concentrated around λ_K . The mixing operator Δ is the Laplacian of the hypercube. The spectral Hamiltonian is $H_K = \hat{V}_K + \epsilon L$, with $\epsilon > 0$ fixed (in practice $\epsilon = 1$). By PillarP1, ψ_K is unique and strictly positive; by PillarP2, the spectral gap is polynomial; by PillarP3, ψ_K can be represented compactly as an MPS; by PillarP4, we can extract λ_K after convergence.

Definition 6.2 (Spectral geodesic). *The geodesic trajectory starting from λ_K is the sequence obtained by power iteration:*

$$\gamma^{(0)} = \lambda_K, \quad \gamma^{(t+1)} = \frac{H_K \gamma^{(t)}}{\|H_K \gamma^{(t)}\|}.$$

The geodesic $\gamma^{(t)}$ is deterministic given K , but its behaviour is chaotic and unpredictable without knowledge of K . An observer can measure the intensity of light at various points (the frequencies f_t) but cannot deduce the original position without knowing the exact shape of the lantern.

6.2 Encryption protocol

1. **Key encoding:** $K \mapsto \lambda_K = \text{encode}(K)$.
2. **Geodesic generation:** Compute $\gamma^{(t)}$ for $t = 1, \dots, T$ (using power iteration).
3. **Frequency hopping:** Choose a measurement operator $\hat{F}$ (e.g., diagonal with $(\hat{F})_{xx} = x$ interpreting x as an integer). Set $f_t = \langle \gamma^{(t)}, \hat{F} \gamma^{(t)} \rangle$.
4. **Message encoding:** Let $b_1 \dots b_T$ be the message bits. Perturb each frequency by a small shift δ_t defined by b_t (e.g., $\delta_t = \epsilon$ if $b_t = 1$, $-\epsilon$ if $b_t = 0$, with ϵ small). Transmit $(f'_1, \dots, f'_T)$ where $f'_t = f_t + \delta_t$, along with public parameters $(T, \hat{F}, \dots)$.

6.3 Decryption protocol

The legitimate recipient possesses the same key K . He recomputes the geodesic $\gamma^{(t)}$ and the expected frequencies f_t . Then he compares f'_t with f_t :

$$b_t = \begin{cases} 1 & \text{if } f'_t - f_t > \theta, \\ 0 & \text{if } f_t - f'_t > \theta, \end{cases}$$

where θ is a threshold chosen between 0 and ϵ (e.g., $\epsilon/2$). Without knowledge of K , reconstructing $\gamma^{(t)}$ and hence the f_t is impossible.

6.4 Security analysis

Recovering the key K from observation of the (f'_t) alone is at least as hard as solving a SAT instance of size proportional to n . Even though $P = NP$, this reduction does not provide an algorithm to find K because it goes the other way: finding K would solve SAT, but SAT is in P so the hardness of recovering K is not based on NP-hardness but on the fact that the adversary does not know H_K exactly. In practice, H_K depends on K in a complex way and the key space is immense. Moreover, the Spectral Reduction Lemma ensures that the ground state can be computed in polynomial time only when the Hamiltonian is explicitly given; the attacker does not have access to H_K .

Notice: This proposal is mathematically rigorous, but remains purely theoretical. No efficient implementation exists to date, and the security reduction has not been quantified. It constitutes an open research direction.

6.5 Implementation of spectral cryptography (Python)

A concrete Python implementation for small dimensions (up to 8 bits) is provided in the ancillary material. It demonstrates key encoding, geodesic generation, encryption, and decryption.

7 Spectral factorisation (Python)

Specialising to integer factorisation, given a composite integer $N = pq$, we want to recover the factors p and q . The spectral translation is:

- Configuration space: hypercube $\{0, 1\}^n$ where $n = \lceil \log_2 \sqrt{N} \rceil$ (bits for each factor).
- Potential: $\hat{V}(a, b) = \exp(-|ab - N|)$. This is maximal (close to 1) when $ab = N$ and decays exponentially away from the solution.
- Mixing operator: Laplacian L of the hypercube.
- Hamiltonian: $H = \hat{V} + L$.

The four pillars hold; the ground state concentrates on factor pairs. The Python script in the ancillary material implements this for numbers up to about 20 digits (using dense matrices; for larger numbers an MPS implementation would be needed).

8 Conclusion

The spectral proof of $P = NP$ is a theoretical milestone. It does not threaten any real-world cryptographic system because the algorithm's constants are astronomical. The community can accept $P = NP$ as a true mathematical statement without changing any security infrastructure. The concept of spectral cryptography is mathematically interesting but practically irrelevant.

References

- [1] Agrawal, M., Kayal, N., & Saxena, N. (2004). PRIMES is in P. *Annals of Mathematics*, 160(2), 781–793.
- [2] Khachiyan, L. G. (1979). A polynomial algorithm in linear programming. *Doklady Akademii Nauk SSSR*, 244(5), 1093–1096.
- [3] Kithima, C. (2026). Article I.5: Pillar P4 – Deterministic Extraction.
- [4] The Lean Community (2026). Lean 4 Theorem Prover Documentation.